

StockNet - Parallelized Neural Network for Stock Price Prediction Using High-Performance Computing Resources

Benjamin Zheng

Henry Odza

Prishita Ghanathe Krishna

Ronald Han

Toby Cowles

Abstract

In this project, we aim to design and implement a stock price predictor using the HPC cluster. Our approach will focus on the S&P 500 historical dataset, which provides a good source of financial data for model training and evaluation. We will begin by collecting and preprocessing this data to make sure it is clean, consistent, and suitable for modeling. Using the computational power of the Markov cluster, we will develop and train multiple neural network models to predict stock price movements and identify stocks likely to outperform market trends. These models will be designed to capture nonlinear relationships and temporal dependencies within the market.

1 Introduction

Financial markets exhibit complex, nonlinear dynamics with strong temporal dependencies, making accurate stock price prediction a challenging modeling problem. In this project we develop StockNet, a neural network framework that combines sequence models (LSTM) and convolutional architectures (1D and 2D CNNs) with high-performance GPU computing on the Markov cluster at CWRU. Our objective is to evaluate how different deep learning architectures perform on S&P 500 stock data and to quantify the benefits of GPU acceleration and multi-GPU parallelization for both training time and predictive accuracy.

2 Datasets and Preprocessing

2.1 Datasets

Our analysis utilizes historical stock price data from the S&P 500 index, obtained from the Kaggle Stock Market Dataset. The dataset contains daily trading records for 500 stocks spanning multiple years, providing a comprehensive view of large-cap U.S. equities.

For each stock $s \in \{1, 2, \dots, S\}$ where S is the total number of stocks, we extract time series data consisting of:

$$\mathbf{x}_s^{(t)} = [L_t, O_t, V_t, H_t, C_t, A_t]^T \in \mathbb{R}^6$$

where for each trading day t :

- (1) L_t : Low price
- (2) O_t : Open price
- (3) V_t : Trading volume
- (4) H_t : High price
- (5) C_t : Close price

- (6) A_t : Adjusted close price

2.2 Preprocessing

To ensure temporal consistency across training samples, we impose a strict completeness requirement. For a stock s to be included in the training set, its data must satisfy:

$$\text{valid}(s) = \begin{cases} 1 & \text{if } |T_s| \geq T_{\min} \text{ and } \nexists t \in [1, T_s] : \mathbf{x}_s^{(t)} = \text{NaN} \\ 0 & \text{otherwise} \end{cases}$$

where T_s denotes the number of available time steps for stock s and $T_{\min} = 1000$ is our minimum sequence length requirement.

This filtering criterion serves two purposes:

- (1) **Temporal Consistency**: Ensures all stocks have equal sequence lengths for batch processing
- (2) **Data Integrity**: Eliminates stocks with missing values that would introduce bias

After applying this filter, our dataset reduces from 500 to S_{valid} stocks, where:

$$S_{\text{valid}} = \sum_{s=1}^S \text{valid}(s)$$

For each valid stock s , we extract the most recent $T = 1000$ consecutive trading days, creating a standardized input representation:

$$X_s = [\mathbf{x}_s^{(1)}, \mathbf{x}_s^{(2)}, \dots, \mathbf{x}_s^{(T)}] \in \mathbb{R}^{T \times 6}$$

This yields a corpus of stock sequences:

$$\mathcal{X} = \{X_1, X_2, \dots, X_{S_{\text{valid}}}\}$$

where each X_s represents a complete 1000-day trading history for stock s .

Raw stock prices exhibit high variance across different equities due to fundamental value differences (e.g., penny stocks vs. high-value stocks). To stabilize training and ensure gradient flow, we apply feature-wise min-max normalization. For feature dimension $f \in \{1, 2, \dots, 6\}$, we first aggregate all values across stocks and time:

$$\mathbf{v}_f = \{X_{s,t,f} : s \in [1, S_{\text{valid}}], t \in [1, T]\}$$

The normalization transformation is then:

$$X_{s,t,f}^{\text{norm}} = \frac{X_{s,t,f} - \min(\mathbf{v}_f)}{\max(\mathbf{v}_f) - \min(\mathbf{v}_f)}$$

This maps all feature values to the range $[0, 1]$, ensuring:

$$X_{s,t,f}^{\text{norm}} \in [0, 1] \quad \forall s, t, f$$

All of this is done within a DataLoader object that is then passed onto the different models for use during training and evaluation.

3 Model Design and Implementation

3.1 Long Short Term Memory (LSTM) Network

One of the architectures we used to implement StockNet is the Long Short Term Memory (LSTM) network. LSTMs are a type of recurrent neural network (RNN) that are well-suited for sequence prediction problems due to their ability to capture long-term dependencies in data. In our implementation, we designed an LSTM network with multiple layers to effectively learn from the historical stock price data.

The complete architecture consists of:

- (1) A multi-layer LSTM encoder that processes historical price sequences
- (2) A fully connected output layer that maps LSTM hidden states to price predictions
- (3) An iterative forecasting mechanism that generates future price sequences based on previous predictions

Our implementation employs a stacked LSTM architecture with L layers, where $L = 2$ in the default configuration. For layer $\ell \in \{1, 2, \dots, L\}$, the hidden state at time t is denoted $h_t^{(\ell)}$. The output of layer $\ell - 1$ serves as input to layer ℓ :

$$h_t^{(\ell)} = \text{LSTM}^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}, c_{t-1}^{(\ell)})$$

where $h_t^{(0)} = x_t$ represents the input sequence. The hidden size is set to $h = 64$ neurons per layer.

To prevent overfitting, we apply dropout regularization between LSTM layers. During training, dropout randomly sets a fraction p of hidden state activations to zero:

$$h_t^{(\ell)} \leftarrow h_t^{(\ell)} \odot m^{(\ell)}$$

where $m^{(\ell)} \sim \text{Bernoulli}(1 - p)$ and $p = 0.2$ in our configuration.

At inference time, activations are scaled by $(1 - p)$ to maintain expected values. The LSTM's final hidden state $h_T^{(L)}$ at the last time step T is transformed to a scalar price prediction via a fully connected layer:

$$\hat{y}_{T+1} = W_{\text{out}} \cdot h_T^{(L)} + b_{\text{out}}$$

where $W_{\text{out}} \in \mathbb{R}^{1 \times h}$ and $b_{\text{out}} \in \mathbb{R}$ are learnable parameters. This yields the predicted normalized close price for the next time step. Raw stock price data undergoes feature-wise min-max normalization to stabilize training:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

where $x_{\min}$ and $x_{\max}$ are computed across all stocks and time steps in the training set. Normalized values lie in $[0, 1]$. We extract only the close price feature for autoregressive modeling, resulting in

input sequences $X \in \mathbb{R}^{N \times (T-1) \times 1}$ where N is the number of stocks and $T = 1000$ is the sequence length.

For our loss function, we employ Mean Squared Error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B (y_i - \hat{y}_i)^2$$

where B is the batch size, y_i is the true normalized close price, $\hat{y}_i$ is the prediction, and θ represents all model parameters. The main idea being that MSE penalizes larger errors, encouraging the model to produce more accurate price predictions.

The Adam optimizer was used with learning rate $\alpha = 0.001$:

$$\theta_{t+1} = \theta_t - \alpha \frac{m_t}{\sqrt{v_t} + \epsilon}$$

where m_t and v_t are exponentially weighted moving averages of gradients and squared gradients, respectively. The training process iterates for $E = 1000$ epochs with batch size $B = 32$. We shuffle training samples at each epoch to reduce overfitting to batch ordering.

As for evaluation, forecasting k steps ahead, we employ an iterative autoregressive approach. Starting with historical sequence $\mathbf{x} = [x_1, x_2, \dots, x_T]$, we predict:

- (1) $\hat{x}_{T+1} = f_{\theta}(\mathbf{x})$
- (2) $\hat{x}_{T+2} = f_{\theta}([x_2, x_3, \dots, x_T, \hat{x}_{T+1}])$
- (3) $\hat{x}_{T+k} = f_{\theta}([x_k, x_{k+1}, \dots, x_T, \hat{x}_{T+1}, \dots, \hat{x}_{T+k-1}])$

where f_{θ} represents the trained LSTM model. This sliding window approach maintains constant input sequence length while incorporating previously generated predictions.

Final predictions are transformed back to original price scale:

$$\hat{y}_{\text{price}} = \hat{y}_{\text{norm}} \times (x_{\max} - x_{\min}) + x_{\min}$$

This allows direct interpretation of forecasted prices in currency units and facilitates comparison with actual market values.

3.2 Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) provide an alternative to recurrent models for sequence modeling by exploiting local temporal structure with convolutional filters. In our work we implement two variants: a 1D temporal CNN that operates along the time axis and returns one prediction per timestep (many-to-many), and a 2D CNN that treats time and feature dimensions as two spatial axes to explicitly model cross-feature interactions at each time window. Both variants consume the normalized input sequences $X \in \mathbb{R}^{T \times F}$ (with $T = 1000$ and $F = 6$ features per time step) and produce a predicted normalized close price for each time step in the output sequence.

3.2.1 1D Convolutional Model. The 1D model applies temporal convolutional filters across the sequence for each feature channel. Intuitively, a 1D convolution with kernel size k slides a filter over time and learns local temporal patterns such as short-term trends,

local maxima/minima, or momentum that are predictive of the next close price. In our implementation the network expects inputs shaped as (batch, T, F) which are permuted to (batch, F, T) before feeding into PyTorch’s Conv1d layers. The architecture is a small stack of Conv1d blocks with ReLU activations and dropout for regularization, followed by a 1×1 projection to produce a single scalar per time step. Formally, if $\mathbf{x}_{b,:t} \in \mathbb{R}^F$ is the feature vector for batch index b at time t , the network computes:

$$\mathbf{h}_{b,:t}^{(1)} = \text{ReLU}(W^{(1)} * \mathbf{x}_{b,:t} + b^{(1)}),$$

where $*$ denotes a 1D convolution over the temporal axis and subsequent layers further mix temporal context. The final projection yields $\hat{y}_{b,t} \in \mathbb{R}$ for each t .

Key training and design choices for the 1D model:

- Many-to-many prediction: the model outputs a sequence $\hat{Y} \in \mathbb{R}^{T \times 1}$ so that intermediate predictions can be used for sequence-level losses and diagnostics. This stabilizes learning relative to single-step forecasting in some regimes.
- Loss and optimizer: Mean Squared Error (MSE) is used as the training objective and optimized with Adam ($\alpha = 10^{-3}$), matching the LSTM’s configuration to allow fair comparison.
- Regularization: dropout is applied after each activation and the receptive field is controlled by kernel size and stacking depth. Padding is used to preserve sequence length so outputs align with inputs.
- Checkpointing: the trainer saves the model state when validation (or training) loss improves, enabling early stopping and safe re-summation of training.

The 1D CNN is computationally efficient and benefits from parallelization on GPUs because convolutions are highly optimized. Its inductive bias favors temporal locality, making it especially effective at learning short-to-medium range temporal patterns with a small parameter count.

3.2.2 2D Convolutional Model. The 2D variant treats the sequence as a 2D image with axes time $\times$ features. Concretely, an input (batch, T, F) is reshaped to $(\text{batch}, 1, T, F)$ and processed by Conv2d layers with kernels (k_t, k_f) that span both temporal and feature neighborhoods. After several Conv2d blocks the network projects to a single channel and reduces the feature axis (for example by averaging) so that the final output shape is $(\text{batch}, T, 1)$.

This design explicitly models interactions between features at local temporal windows, enabling the network to learn patterns such as joint movements of open/high/low/close/volume that are predictive of future close prices. The forward pass can be summarized as:

$$\mathbf{Z} = \text{Conv2D}_{\text{stack}}(\text{reshape}(X)), \quad \hat{Y}_{b,t} = \text{pool}_f(\mathbf{Z}_{b,1,t,:}),$$

where pool_f denotes the operation that collapses the feature axis (e.g., mean over features) after the final 1-channel projection.

Advantages and trade-offs of the 2D model:

- Cross-feature coupling: 2D kernels can learn filters that respond to specific combinations of features (for instance, a high-volume up-tick pattern), which the 1D model can only approximate indirectly by combining per-feature channels.
- Larger parameter and compute cost: Conv2d layers typically require more memory and compute than Conv1d for the same effective receptive field, especially when using larger feature kernels k_f .

This increases GPU memory usage and may slow training for long sequences like $T = 1000$.

- Flexible receptive field: by tuning (k_t, k_f) and stacking depth we can control the temporal and cross-feature scope of learned filters. Multi-scale kernels or separable convolutions are straightforward extensions to reduce cost while preserving expressivity. Both models are trained on the same normalized dataset and evaluated with identical metrics (MSE on held-out validation sequences) so that their statistical and computational trade-offs can be directly compared. In practice, the 1D model often trains faster and requires fewer parameters for comparable short-range forecasting accuracy, while the 2D model can outperform when cross-feature patterns are strong and worth the additional computational expense.

Practical improvements to consider (low-risk extensions): dilated convolutions or residual blocks to increase effective temporal context without proportional parameter growth; depthwise separable convolutions to reduce cost of 2D filters; and multi-scale branches that combine short and long kernels to capture patterns at multiple horizons. These ideas are easy to prototype within the existing trainer and model modules and can yield robust gains for stock time-series, which exhibit structure at many temporal scales.

4 Parallelization Strategy

The central goal of StockNet is to exploit the GPU resources of the Markov cluster to accelerate training for all three architectures while preserving a unified codebase for single- and multi-GPU runs. The implementation separates model logic from HPC orchestration, uses PyTorch’s Distributed Data Parallel (DDP) framework for intra-node multi-GPU training, and logs all runs in a reproducible way through structured checkpoints and results files. All GPU jobs are executed on the Markov GPU partition with NVIDIA GPUs, using the module PyTorch/2.1.2-foss-2023a-CUDA-12.1.1. A project-specific Python virtual environment is created in storage space and activated at job start. SLURM scripts are stored in `slurm_scripts/`, model checkpoints in `checkpoints/`, logs in `logs/`, and metrics and summaries in `results/`.

4.1 Single-GPU training pipeline

Single-GPU experiments for the LSTM, 1D CNN, and 2D CNN architectures are driven by a shared entry point `main.py`. This script provides a simple command-line interface that selects the model and training configuration while keeping optimization hyperparameters aligned. A typical single-GPU run for the 1D CNN model is launched with:

```
python main.py --mode train --model cnn
--cnn_type 1d --epochs 1000
--batch_size 64 --device cuda
```

Internally, `main.py` constructs either an LSTMTrainer or a CNNTrainer and delegates the full training loop to these trainer classes. Both trainers rely on a common preprocessing utility that loads the S&P 500 dataset, enforces a minimum sequence length of $T = 1000$ trading days, and applies feature-wise min-max normalization to obtain tensors $X \in \mathbb{R}^{N \times T \times F}$ with aligned targets y .

On Markov, single-GPU runs are wrapped in dedicated SLURM scripts for each architecture. For example, we use separate scripts for the LSTM, CNN-1D, and CNN-2D models, each requesting one GPU, eight CPU cores, and 32 GB of memory:

```
#SBATCH --partition=markov_gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=02:00:00
```

Each job logs metadata (hostname, job ID) as well as GPU type and memory capacity into the log directory. During training, the selected model is moved to cuda and optimized for up to 1000 epochs with batch size 64 and the Adam optimizer, using mean squared error as the loss. The best-performing weights are saved in model-specific checkpoint files and reused for evaluation and inference. These single-GPU runs provide both strong baselines and the reference timings T_1 used in the empirical speedup calculations.

4.2 Multi-GPU distributed training with DDP

To scale training beyond a single device, multi-GPU data parallelism is implemented in a dedicated script that uses PyTorch’s Distributed Data Parallel (DDP) API. The script launches one worker process per GPU via `torch.multiprocessing.spawn`, with each process identified by a rank $0 \leq r < \text{world_size}$. The DDP environment is initialized by forming an `nccl` process group and binding each rank to a dedicated device:

```
dist.init_process_group(...,
    world_size=world_size, rank=rank)
torch.cuda.set_device(rank)
```

Each rank constructs its own preprocessing object and obtains the normalized dataset. The resulting tensors are wrapped into a custom distributed dataset and fed through a data loader that uses PyTorch’s distributed sampler, so that each GPU receives a distinct shard of the data while global shuffling is preserved across epochs. For the model, each rank instantiates the chosen architecture (LSTM, CNN-1D, or CNN-2D), moves it to the local device, and wraps it in DDP:

```
model = DDP(model.to(rank), device_ids=[rank])
```

Training then proceeds with the same mean squared error loss and Adam optimizer as in the single-GPU case. At every backward pass, DDP performs an all-reduce over gradients so that each replica maintains synchronized parameters. The script records per-epoch losses and wall-clock times, and on rank 0 additionally reports model size, effective batch size, and basic GPU memory usage. Because the Markov scheduler did not reliably provide four-GPU allocations for this project, multi-GPU experiments are restricted to two GPUs on a single node, using the one-GPU runs as baselines for speedup and efficiency computation.

4.3 SLURM orchestration and resource configuration

Multi-GPU jobs are orchestrated through SLURM batch scripts that extend the single-GPU template to request two GPUs and additional

CPU and memory resources. A representative example is the script used for the 1D CNN model:

```
#SBATCH --partition=markov_gpu
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --gres=gpu:2
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --time=02:00:00

python train_distributed.py \
    --model cnn --cnn_type 1d \
    --epochs 1000 --batch_size 64 \
    --world_size 2
```

An analogous script named `train_cnn2d_2gpu.slurm` controls the 2D CNN experiments, differing only in the value of the `cnntype2D` flag. Each job records GPU names and total memory, followed by detailed training output such as loss curves, epoch times, and configuration flags in a dedicated log file. The LSTM architecture is evaluated only in the single-GPU regime, since its training time on one GPU is already modest and two-GPU configurations did not provide a clear advantage under Markov constraints.

4.4 Metrics, checkpointing, and inference benchmarking

To support the analysis in the Experiments and Results sections, the training scripts export structured metrics as JSON and CSV summaries under `results/`. For each model and GPU configuration, these summaries record total training time, per-epoch duration, final loss, effective batch size, and training throughput in samples per second. Using the 1-GPU runs as reference, we compute empirical speedup $S(N) = T_1/T_N$ and parallel efficiency $S(N)/N$. For example, the CNN-1D training time decreases from 41.0 s on one GPU to 23.6 s on two GPUs, yielding a 1.74× speedup and 86.8 % efficiency, while CNN-2D improves from 153.0 s to 75.1 s with a 2.04× speedup and 101.9 % efficiency.

After training completes, `train_distributed.py` also performs an inference benchmark on rank 0. Following a warmup phase, the script measures forward-pass latency for repeated single-sample and batched inputs and reports mean and percentile latencies together with throughput in samples per second. These statistics are stored alongside the training metrics and are used in the results section to characterize deployment-time behavior. In particular, the measured inference throughputs for the CNN-1D and CNN-2D models reach approximately 41 336 and 9 866 samples per second, respectively, on Markov GPUs.

5 Experiments

To evaluate the performance and scalability of our StockNet models, we conducted a comprehensive series of experiments across different computational configurations. Our experimental design focuses on understanding how model training scales with varying hardware resources, comparing both GPU-accelerated and CPU-based training approaches. The experiments were executed on the

Markov HPC cluster, utilizing SLURM workload manager for resource allocation and job scheduling.

5.1 Experimental Setup

Our experimental framework encompasses three distinct model architectures: the LSTM network described in Section 3.1, a 1D Convolutional Neural Network (CNN1D), and a 2D Convolutional Neural Network (CNN2D). For each architecture, we systematically varied the computational resources to investigate scaling behavior and training efficiency. The experimental configurations span single GPU training for all three architectures and multi-GPU distributed training with two GPUs for the CNN models, while separate CPU-only baselines are run as part of the overall study.

All GPU experiments were conducted on the `markov_gpu` partition with NVIDIA GPUs, allocating 8 CPU cores and 32GB of memory per job. Multi-GPU experiments utilized PyTorch’s Distributed Data Parallel (DDP) framework to enable efficient parallel training across multiple devices. Each training run was configured for a maximum of 1000 epochs with early stopping criteria, using a batch size of 64 for single GPU experiments and scaled batch sizes for multi-GPU configurations to maintain computational efficiency.

For each configuration, the training scripts record wall-clock time, per-epoch duration, and model accuracy metrics in structured JSON and CSV files under the `results/` directory. These summaries are later aggregated into performance tables that report training time, empirical speedup, parallel efficiency, and prediction quality for each architecture and GPU count.

5.2 SLURM Job Configurations

To manage the diverse experimental conditions, we developed a suite of SLURM batch scripts that automate the training process across different resource allocations. Each script follows a standardized template that specifies job parameters, loads required modules, activates the Python environment, and executes the training script with appropriate configuration flags. For example, our single GPU training script for the 1D CNN model includes:

```
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G

python main.py --mode train --model cnn
               --cnn_type 1d --epochs 1000
               --batch_size 64 --device cuda
```

Multi-GPU configurations extend this template by requesting additional GPU resources (`-gres=gpu:N`) and invoking PyTorch’s distributed launcher:

```
torchrun --nproc_per_node=N main.py
--distributed ...
```

where $N \in \{2, 4\}$ specifies the number of GPUs. This distributed setup partitions the dataset across available GPUs, with each device processing a subset of the batch in parallel. Gradient synchronization occurs at each optimization step through all-reduce operations, ensuring model consistency across workers.

5.3 CPU Training Experiments

In addition to GPU-accelerated training, we conducted CPU-only experiments to establish performance baselines and understand the computational advantages provided by GPU hardware. CPU training jobs were submitted to standard compute partitions without GPU allocation, utilizing higher core counts (16-32 cores) to leverage multi-threading capabilities. These experiments are particularly valuable for:

- (1) Quantifying the speedup achieved through GPU acceleration
- (2) Assessing the feasibility of model training in resource-constrained environments
- (3) Identifying computational bottlenecks that benefit most from parallel processing

CPU experiments used identical hyperparameters and dataset configurations as their GPU counterparts, differing only in the device specification (`-device cpu`). Training times and memory utilization were recorded for direct comparison with GPU results.

5.4 Experimental Metrics

For each experimental configuration, we collected comprehensive performance metrics including:

- **Training Time:** Wall-clock time to complete the full training regimen
- **Epoch Duration:** Average time per training epoch across all iterations
- **GPU Utilization:** Device memory usage and compute utilization percentages
- **Scaling Efficiency:** Speedup ratio when moving from 1 to N GPUs, defined as $S(N) = \frac{T_1}{T_N}$ where T_N is training time with N GPUs
- **Model Accuracy:** Mean squared error on validation set across training checkpoints

All experiments were logged systematically with job-specific output files (`logs/model_Ngpu_%j.out`) containing training progress, loss curves, and resource utilization statistics. This logging infrastructure enables post-hoc analysis of convergence behavior and identification of optimal hardware configurations for each model architecture.

The complete experimental suite comprises 13 distinct job configurations spanning the three model types, the available GPU configurations (one and two GPUs), and CPU baselines, together with additional retraining runs for model refinement. This comprehensive approach allows us to make evidence-based recommendations for resource allocation when deploying stock prediction models on HPC infrastructure.

6 Results

This section summarizes the empirical impact of GPU acceleration and multi-GPU training on both runtime and predictive performance for the LSTM, CNN-1D, and CNN-2D architectures. We first report end-to-end training time and accuracy metrics for each model-GPU configuration, and then discuss observed speedups, parallel efficiency, and inference throughput on the Markov cluster.

6.1 Training time and accuracy

Table 1 reports the observed training times and prediction quality for each architecture and GPU count. The metrics are computed on a held-out test split of the S&P 500 dataset and include the coefficient of determination R^2 , root mean squared error (RMSE), mean absolute error (MAE), and directional accuracy (percentage of days for which the sign of the next-day return is predicted correctly).

Model	GPUs	Time	R^2	RMSE	MAE	Dir. Acc.
LSTM	1	76.0	0.9774	0.000285	0.000145	97.5
CNN 1D	1	41.0	0.9767	0.000273	0.000231	62.4
CNN 1D	2	23.6	0.8967	0.000575	0.000424	62.7
CNN 2D	1	153.0	0.9702	0.000309	0.000251	62.4
CNN 2D	2	75.1	0.8522	0.000687	0.000584	62.7

Table 1: Training time (seconds) and prediction quality for each architecture and GPU configuration. Dir. Acc. is the percentage of test days for which the model correctly predicts the sign of the next-day return.

These results show that all three architectures achieve strong predictive performance in the single-GPU regime, with R^2 values around 0.97 and very small RMSE and MAE. The LSTM achieves the highest directional accuracy at 97.5%, indicating that it almost always predicts the correct direction of the next-day move on the test set. The CNN models maintain directional accuracy around 62–63%, which still exceeds the 50% random baseline and suggests that convolutional architectures capture useful short-term patterns in the normalized price sequences.

Moving from one to two GPUs substantially reduces training time for the CNN architectures, but at the cost of a moderate drop in predictive accuracy. For CNN-1D, training time decreases from 41.0 s to 23.6 s when scaling from one to two GPUs, while R^2 falls from 0.9767 to 0.8967 and both RMSE and MAE roughly double. A similar pattern appears for CNN-2D, where training time falls from 153.0 s to 75.1 s but R^2 decreases from 0.9702 to 0.8522 and the error metrics increase. These trends indicate that aggressive parallelism shortens wall-clock training but may require additional regularization or hyperparameter tuning to fully preserve single-GPU accuracy.

6.2 Speedup and inference performance

The aggregated GPU performance metrics provide a more detailed view of scaling efficiency. For the CNN-1D model, moving from one to two GPUs yields a 1.74× speedup relative to the single-GPU baseline, corresponding to a parallel efficiency of 86.8 %. The CNN-2D model achieves a 2.04× speedup with two GPUs, and a measured efficiency of 101.9 %, which reflects better utilization of compute resources when the heavier 2D convolutional layers are distributed across devices. These speedups confirm that the DDP-based parallelization strategy effectively reduces training time for convolutional architectures on Markov.

Beyond training, the distributed script performs an inference benchmark on the final models to evaluate deployment-time throughput. On Markov GPUs, the CNN-1D model processes approximately

41 336 samples per second, while the CNN-2D model reaches around 9 866 samples per second under the batched benchmark configuration. Taken together with the accuracy results, these numbers indicate that the GPU-accelerated StockNet models not only train efficiently on the cluster but also support high-throughput prediction, making them suitable for large-scale backtesting or real-time screening workloads.

7 Discussion and Conclusion

7.1 Summary of findings

7.2 Limitations

7.3 Future work

8 References

- [1] PyTorch Documentation. “Distributed data parallel.” Available at <https://pytorch.org/docs>.
- [2] Kaggle. “S&P 500 stock data.” Available at <https://www.kaggle.com/>.